SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	B.Shyam raj	**Reg. No.:**	192421176
Section / Batch:	2024	**Date:**	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – APPLICATION BASED QUESTIONS

Q1.Blockchain Transaction Verification System

A blockchain system verifies transactions using the recurrence T(n) = 5T(n/5) + log n. Apply the Master Theorem to determine the time complexity. Identify parameters and justify the case selection. Analyze how performance changes as transaction volume increases. Interpret efficiency using asymptotic notation.

Q2.Software Log Sorting System

One log sorting algorithm takes O(n log n) time, while another takes O(n²) time. Compare both algorithms using asymptotic notation, explain differences in growth rates, and decide which is better for large datasets. Also compare their space complexities and identify the more memory-efficient choice.

Code of Conduct

I _ __ B.Shyam raj (192421176)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Blockchain Transaction Verification System

A blockchain system verifies transactions using the recurrence T(n) = 5T(n/5) + log n. Apply the Master Theorem to determine the time complexity. Identify parameters and justify the case selection. Analyze how performance changes as transaction volume increases. Interpret efficiency using asymptotic notation.

Introduction

Blockchain is a distributed ledger technology used to securely record and verify transactions across multiple computers. Every transaction must be validated before it is permanently stored in the blockchain.Algorithm analysis helps us understand how the execution time changes as the input size grows.

The given recurrence relation,

$$\boldsymbol{T(n) = 5T(n/5) + \log n}$$

represents a divide-and-conquer algorithm in which a problem of size n is divided into 5 equal subproblems, each of size n/5. After solving these subproblems recursively, an additional log n amount of work is required to combine the results.

To determine the running time, the Master Theorem is applied.

Master Theorem

The Master Theorem is used to solve recurrence relations of the form

$$\boldsymbol{T(n) = aT(n/b) + f(n)}$$

where

- a = Number of recursive subproblems
- b = Factor by which the problem size is reduced
- f(n) = Cost of dividing and combining the subproblems

The theorem provides three cases for determining the asymptotic running time.

Step 1: Identify the Parameters

Comparing the given recurrence with the standard form,

$$T(n) = 5T(n/5) + \log n$$

we obtain

- a = 5
- b = 5
- f(n)=\log n

Step 2: Calculate $n^{\log_b a}$

Using the formula,

$$n^{\log_b a}$$

Substituting the values,$n^{\log_5 5}$

Since,$\log_5 5 = 1$

we get,$n^{\log_b a} = n$

Step 3: Compare $f(n)$with $n^{\log_b a}$
Here,,$f(n) = \log n$

And,$n^{\log_b a} = n$

The logarithmic function grows much more slowly than the linear function.
Mathematically,,$\log n = O(n^{1-\varepsilon})$

for some constant,$\varepsilon > 0$

Therefore,$f(n)$

is polynomially smaller than,$n^{\log_b a}$

Step 4: Select the Appropriate Case
Since,$f(n) = O(n^{1-\varepsilon})$

the recurrence satisfies Case 1 of the Master Theorem.
Case 1 states that
If,$f(n) = O(n^{\log_b a - \varepsilon})$

Then,$T(n) = \Theta(n^{\log_b a})$

Hence,,$T(n) = \Theta(n)$

Time Complexity
The final time complexity is,$\boxed{\Theta(n)}$

which means the running time increases linearly with the number of transactions.

Performance Analysis
The blockchain verification system divides the total transactions into five equal groups and processes them recursively. After processing all groups, only a small amount of additional work, equal to log n, is required.
As the number of transactions increases:

- The recursive calls increase proportionally.
- The additional logarithmic work remains comparatively very small.
- Therefore, the overall running time is dominated by the recursive computation.

For example,

Number of Transactions	Approximate Running Time
100	Proportional to 100
500	Proportional to 500
1000	Proportional to 1000
5000	Proportional to 5000

Thus, if the number of transactions doubles, the execution time approximately doubles. This shows excellent scalability for large blockchain networks.

Interpretation Using Asymptotic Notation

Big-O Notation

Big-O represents the maximum running time.

$$T(n) = O(n)$$

The algorithm will never take more than linear time.

Big-Theta Notation

Big-Theta represents the exact growth rate.

$$T(n) = \Theta(n)$$

The running time grows exactly in proportion to the number of transactions.

Big-Omega Notation

Big-Omega represents the minimum running time.

$$T(n) = \Omega(n)$$

Even in the best case, every transaction must be processed at least once.

Advantages of Linear Time Complexity

- Efficient for large blockchain networks.
- Scales well as transaction volume increases.
- Suitable for real-time verification.
- Predictable performance.
- Reduces processing delays in distributed systems.

Conclusion

The recurrence relation

$$T(n) = 5T(n/5) + \log n$$

matches the standard form of the Master Theorem with

- a = 5
- b = 5
- f(n)=\log n

Since

$$\log n = O(n^{1-\varepsilon}),$$

Case 1 of the Master Theorem applies.
Therefore,

$$\boxed{T(n) = \Theta(n)}$$

The algorithm has linear time complexity, meaning its running time increases proportionally with the number of blockchain transactions. This makes the verification system efficient, scalable, and suitable for handling large transaction volumes.

Q2. Software Log Sorting System

One log sorting algorithm takes O(n log n) time, while another takes O(n²) time. Compare both algorithms using asymptotic notation, explain differences in growth rates, and decide which is better for large datasets. Also compare their space complexities and identify the more memory-efficient choice.

Introduction

Software systems generate large numbers of log files containing information about user activities, system events, errors, and security operations. These logs often need to be sorted before analysis. Choosing an efficient sorting algorithm is important because the amount of data may range from a few hundred records to millions of records.
The two algorithms considered are:

- Algorithm A: O(n log n)
- Algorithm B: O(n²)

Their efficiency is compared using asymptotic notation.

Asymptotic Notation

Asymptotic notation describes the performance of an algorithm as the input size becomes very large.
The three common notations are:

- Big-O (O) – Upper bound
- Big-Theta (Θ) – Exact bound
- Big-Omega (Ω) – Lower bound

These notations ignore constants and focus only on the rate at which execution time grows.

Algorithm Comparison
Algorithm A
Time Complexity:

$$O(n\log n)$$

Examples include:

- Merge Sort
- Heap Sort
- Quick Sort (Average Case)

These algorithms divide the input into smaller parts and efficiently combine the sorted results.

Algorithm B
Time Complexity:

$$O(n^2)$$

Examples include:

- Bubble Sort
- Selection Sort
- Insertion Sort (Worst Case)

These algorithms compare elements repeatedly, leading to quadratic growth.

Growth Rate Comparison
The following table compares the number of operations.

Number of Logs (n)	$n\log_2 n$	n^2
100	664	10,000
500	4,483	250,000
1,000	9,966	1,000,000
5,000	61,439	25,000,000
10,000	132,877	100,000,000

From the table, it is clear that the O(n²) algorithm performs dramatically more operations as the input size increases.

Performance Analysis

O(n log n) Algorithm

- Efficient for both small and large datasets.
- Execution time grows slowly.
- Handles millions of records effectively.
- Widely used in modern software systems.

O(n²) Algorithm

- Acceptable only for small datasets.
- Running time increases rapidly.
- Performance becomes poor for large log files.
- Not suitable for large-scale applications.

Which Algorithm is Better?
For large datasets,

$$\boxed{O(n\log n)}$$

is the better choice because

- fewer comparisons are required,
- lower execution time,
- better scalability,
- faster processing,
- improved overall efficiency.

Space Complexity Comparison
Different sorting algorithms require different amounts of memory.

Sorting Algorithm	Time Complexity	Space Complexity
Merge Sort	O(n log n)	O(n)
Heap Sort	O(n log n)	O(1)
Quick Sort (Average)	O(n log n)	O(log n)
Bubble Sort	O(n²)	O(1)
Selection Sort	O(n²)	O(1)

Memory Efficiency
If the O(n log n) algorithm is Merge Sort,

- Extra memory required = O(n)

If the O(n²) algorithm is Bubble Sort or Selection Sort,

- Extra memory required = O(1)

Therefore,
Bubble Sort is more memory-efficient but slower.
However, if the O(n log n) algorithm is Heap Sort,

- Extra memory = O(1)

In this case,
Heap Sort is both faster and equally memory-efficient.

Real-World Applications
O(n log n) Algorithms

- Database indexing
- Software log analysis
- Blockchain transaction ordering
- Cloud computing
- Search engines
- Banking systems

O(n²) Algorithms

- Small classroom programs
- Educational demonstrations
- Very small datasets
- Simple embedded systems

Advantages of O(n log n) Algorithm

- Faster execution
- Better scalability
- Suitable for big data
- Reduced processing time
- Used in industry-standard software

Conclusion
The O(n log n) algorithm is significantly more efficient than the O(n²) algorithm for sorting software logs. Its growth rate is much slower, making it ideal for large datasets and real-world applications.
From the time complexity perspective, O(n log n) is the preferred choice. In terms of space complexity, Bubble Sort or Selection Sort use only O(1) extra memory, while Merge Sort requires O(n). However, Heap Sort achieves both O(n log n) time and O(1) space, making it one of the most balanced choices.
Therefore:

- Best Time Complexity: $\boxed{O(n\log n)}$
- Most Memory-Efficient: $\boxed{O(1)}$ space (Heap Sort, Bubble Sort, or Selection Sort)
- Best Overall Choice for Large Datasets:
  $\boxed{O(n\log n)}$ sorting algorithm (preferably Heap Sort or Merge Sort)